With this rule book that is defined, I want to write a Functional document, so i can create/build a program using python. So create me a functional document. I can provide the prices on excel file as attached. Write me a functional document so that a programmer can build it. I want a ledger book implemented as well. The outcome should It should simulate buy, sell actions of the corresponding stock. Create me Functional document

Below is a **Functional Specification Document** for implementing your Enhanced Leveraged ETF Strategy v2.0 in Python. It is designed to guide a programmer in building a systematic backtesting engine that can read price/volume data from Excel, process your rulebook, and generate a complete trade ledger including buy/sell/hold actions, positions, and outcomes.

Functional Specification Document

Enhanced Leveraged ETF Strategy v2.0

Objective

Develop a Python program that:

- Ingests historical QQQ price and volume data from an Excel file.
- Implements the strategy rules as alerts, simulates trading actions (buy/sell/hold) for TQQQ and SQQQ.
- Produces a trade ledger book recording all simulated transactions and daily positions.

1. Input Specifications

1.1 Data File

- **Accepted Format:** Excel (.xlsx or .csv)
- **Required Columns:**
  - Date
  - QQQ Close Price
  - QQQ High Price
  - QQQ Low Price
  - QQQ Volume

1.2 Parameters (Configurable)

- Initial Capital (Default: $10,000)
- Transaction fee per trade (Default: $0 or user-input)
- Ticker mapping (QQQ, TQQQ, SQQQ)
- Position sizing limits (Max allocation % per trade)
- Buffer percentages (for entry/exit triggers)

2. Technical Indicator Calculations

For each trading day:

- **50-day SMA**: rolling average of last 50 close prices
- **200-day SMA**: rolling average of last 200 close prices
- **SMA Slopes**: percentage change of each SMA over last 5 days
- **14-day ATR**: rolling average of daily true range (max[high-low, high-prev_close, low-prev_close]) over last 14 days
- **20-day Avg Volume**: rolling average of last 20 volumes
- **Volume Ratio**: current volume / 20-day average

3. Rule Engine

Implement each alert as a modular rule function:

3.1 Priority Evaluation (per alert summary):

1. **Death Cross**: 50-SMA < 200-SMA triggers immediate move to 100% cash.
2. **High Volatility**: If ATR% over regime threshold (varies by regime), reduce position or hold cash.
3. **Entry Signals** (TQQQ/SQQQ/cash): Triggered by combined momentum, buffer zones, volume ratio, multi-timeframe alignment, and time-based filters.

4. **Exit Signals**: Triggered by stop-loss, SMA slope deterioration, price breaches exit buffer, or volatility spikes.
5. **Volume Adjustments**: Adjust position size based on volume confirmation.
6. **Time-Based Reductions**: Lower position sizes during holidays, earnings, Fed/event days.
7. **Multi-Timeframe Checks**: Use weekly SMA confirmations for full position sizing.
8. **Slope Acceleration**: Confirm entry only if current SMA slope is accelerating.
9. **Trailing Stops**: If position > 15% gain, set new trailing stop logic.

4. Trade Execution Simulation

- **Buy Action**: Increase the position (TQQQ or SQQQ) according to rule-derived size, deduct capital, account for fees.
- **Sell Action**: Reduce or exit the position per rule triggers, update cash balance.
- **Hold Action**: Maintain current position, update market value based on daily price.

Logic should simulate proper application of position buffers, volume ratios, time filters, and trailing stops as defined in rulebook.

5. Trade Ledger Book

- **Ledger Columns (main table per day):**
  - Date
  - Action (Buy TQQQ, Sell TQQQ, Buy SQQQ, Sell SQQQ, Hold, Move to Cash)
  - Reason/Triggered Rule (e.g., Alert #3: 100% TQQQ, Alert #12: Volume Adjustment)
  - Position Size (%) and ETF (e.g., 50% TQQQ, 50% SQQQ)
  - Price executed
  - Total shares
  - Capital remaining
  - Daily equity value
  - Trailing stop values
  - ATR, SMA slopes, volume ratio, momentum, and which rule triggered
  - Transaction fee
- All trades and transitions should be recorded chronologically for auditing/backtesting.

6. Output & Reporting

- **Trade ledger (CSV or Excel export)**
- **Daily equity curve (portfolio value graph)**
- **Performance metrics**:
  - Total return, CAGR, max drawdown, win rate, profit factor, Sharpe ratio
- **Position distribution by day (% in TQQQ, SQQQ, Cash)**
- **Trade statistics (average hold, total trades, average gain/loss, stop-losses triggered)**
- **Summary table of all rules triggered with counts**

7. Extensibility/Testing

- Design for modular rule addition or changes.
- Should support running on subsets of dates for walk-forward or out-of-sample testing.
- Allow easy changes of parameters or rules for future refinement.
- Built-in sanity check to validate data integrity (no missing dates, prices).

8. Example Workflow

1. **Data Import:** User provides Excel file with 5-year QQQ data.
2. **Config Set:** User sets initial capital, buffer %, transaction fee (defaults used if none).
3. **Indicators Calculated:** On each step, program calculates necessary SMA, ATR, volume, etc.
4. **Rules Applied:** For each day, rules engine triggers buy/sell/hold/cash actions.
5. **Ledger Book Updated:** Each action and daily portfolio summary logged.
6. **Results Exported:** User receives trade ledger and performance report.

9. Additional Notes

- Program should be robust to errors in input data (e.g., skips missing price points with alert).
- Actions (buy/sell) always logged with reference to triggered rule.
- Logic must prioritize rules using hierarchy specified in rulebook (death cross > stop-loss > volatility > entry signal > time/volume adjustment, etc.).
- All calculations strictly follow the formulas and thresholds defined above.

10. User Interfaces (Recommended)

- Command-line interface with config file input (minimum).
- Optional: Interactive Jupyter/Colab notebook template for quick operation and edits.

Supplementary Materials Needed:

- DETAILED RULE SUMMARY (provided above): Attach as part of spec or comments for coder.
- Example input file for QQQ price and volume history.

End Functional Specification — Ready for Programming Implementation.

⁂

1. https://www.perplexity.ai/finance/QQQ
2. https://www.perplexity.ai/finance/TQQQ
3. https://www.perplexity.ai/finance/SQQQ